Blue Harvest Agentic SDLC Engineering Assignment



Scenario

Blue Harvest is supporting a financial-services client that offers mortgages through a network of relationship managers.

The client needs a small internal service to manage **mortgage pricing exception requests**. A relationship manager may request a discount on the standard interest rate for a mortgage application. An authorised reviewer must decide whether to approve or decline the request before it can be used in the application process.

Build a service that lets internal users create a pricing exception request, review it, and retrieve its outcome.

What the client needs

- A relationship manager can create a request for an existing mortgage application. A request includes an application identifier, the requested rate discount (in basis points), and a reason.
- A reviewer can approve or decline a request. The outcome should identify who made the decision and when.
- An approved request must be available to the mortgage application process, which needs to know the discount that was approved.
- Internal users need to see the history of a request, including its current status and previous decisions.
- The client wants to avoid duplicate requests when an upstream system retries a call after a timeout.
- The service should be straightforward to run locally for this exercise. You may represent mortgage applications and users in the way you consider most useful.

One stakeholder says that a relationship manager should be able to change the requested discount after submitting it. Another says that an approval must always apply to the exact request that was reviewed. Treat this as genuine input: make and explain the decision you think is appropriate.

Deliverable

Create a working, runnable solution. An HTTP API is expected, unless you make and explain a different choice. Provide enough seed data or setup instructions for us to exercise a realistic workflow.

- Source code in a Git repository, with instructions to run, test, and explore the solution.
- A concise README explaining the problem you solved, your assumptions, and deliberate scope choices.
- Evidence that important behaviour is validated. Choose the test levels and examples that give you confidence.
- Any decisions, diagrams, notes, or operational guidance that another engineer would need to safely continue the work.

Constraints

- Use any language, framework, storage approach, or open-source tooling. Java or C# are familiar to our team, but not required.
- The solution must be easy for us to run locally. Do not require access to private infrastructure or paid services.
- State what you would change before production use if your exercise implementation makes a lighter trade-off.
- Use agentic development approaches where they add value. Be ready to discuss how AI-assisted tools or agents contributed to your work and how you validated their output.

Bonus

Demonstrate how AI agents can enhance the solution or the SDLC around it. Keep the demonstration grounded: explain the problem it addresses, how you would control or verify the agent's output, and what risks remain. This can be part of the repository rather than a separate product feature.

A simple user interface that lets internal users create, review, and inspect pricing exception requests.

Submission

Share the repository with us and include the time you spent. We do not expect a large system. We value a small, clear solution with thoughtful decisions over a broad but fragile implementation.

We may ask you to walk us through your choices, run a scenario, and describe what you would do next with more time.